

Django Permission System + Race Condition

Part 1: Django Permission System

1. What is Permission?

A **Permission** in Django determines whether a user is allowed to perform a specific action.

Simple idea:

```
javascript.snippet JAVASCRIPT

Authentication Who are you?Authorization What are you allowed to do?Permission
What
specific action can you perform?
```

Example:

```
javascript.snippet JAVASCRIPT

User: FarukPermissions:v View Articlev Add Articlev Change Article Delete Article
```

So, the user can view, create, and edit articles but cannot delete them.

2. Django's Built-in Model Permissions

When you create a model, Django normally creates four default permissions:

```
javascript.snippet JAVASCRIPT

addchangedeleteview
```

For example:

```
javascript.snippet JAVASCRIPT

class Article(models.Model):    title = models.CharField(max_length=200)
```

Django creates permissions similar to:

```
javascript.snippet JAVASCRIPT
add_articlechange_articledelete_articleview_article
```

The full permission name is generally:

```
javascript.snippet JAVASCRIPT
app_label.codename
```

Example:

```
javascript.snippet JAVASCRIPT
blog.add_articleblog.change_articleblog.delete_articleblog.view_article
```

3. Creating Custom Permissions

You can create permissions beyond Django's default permissions.

For example:

```
javascript.snippet JAVASCRIPT
can_publish
```

This means that a user can publish an article.

models.py

The clean approach is to define custom permissions in the model's Meta class:

```
javascript.snippet JAVASCRIPT
from django.db import modelsclass Article(models.Model):    title = models.CharField(
max_length
=200)    content = models.TextField()    published = models.BooleanField(default=False)

class Meta:    permissions = [        ("can_publish", "Can publish articles"),
    ]
```

Then run:

```
javascript.snippet JAVASCRIPT

python manage.py makemigrationspython manage.py migrate
```

Django will create:

```
javascript.snippet JAVASCRIPT

blog.can_publish
```

4. Manually Creating a Permission

You can also create a permission using Django's Permission model.

```
javascript.snippet JAVASCRIPT

from django.contrib.auth.models import Permissionfrom django.contrib.contenttypes.
models
import ContentTypefrom django.contrib.auth.models import Usercontent_type =
ContentType
.objects.get_for_model(User)permission = Permission.objects.create(    codename="
can_publish
",    name="Can Publish Articles",    content_type=content_type,)
```

However, for normal application-level custom permissions, defining them through `Meta.permissions` is usually cleaner and more maintainable.

5. Checking a User's Permission

The most important method is:

```
javascript.snippet JAVASCRIPT

user.has_perm()
```

Example:

```
javascript.snippet JAVASCRIPT

if request.user.has_perm("blog.can_publish"):    # User can publish
```

Or:

```
if not request.user.has_perm("blog.can_publish"):    # Permission denied
```

6. Permission in Function-Based Views

For FBVs, Django provides the `permission_required` decorator.

```
from django.contrib.auth.decorators import permission_requiredfrom django.shortcutsimportrender@permission_required("blog.can_publish", raise_exception=True)defpublish_article(request):    return render(request, "publish.html")
```

Here:

```
permission_required("blog.can_publish")
```

means the user must have the `blog.can_publish` permission.

And:

```
raise_exception=True
```

causes Django to return a permission-denied response instead of simply redirecting to the login page when the user is authenticated but lacks the permission.

7. Permission in Class-Based Views

For CBVs, use `PermissionRequiredMixin`.

```
javascript.snippet JAVASCRIPT

from django.contrib.auth.mixins import PermissionRequiredMixin from django.views.generic

import TemplateView class PublishView( PermissionRequiredMixin, TemplateView):
    template_name
    = "publish.html" permission_required = "blog.can_publish" raise_exception = True
```

URL:

```
javascript.snippet JAVASCRIPT

from django.urls import path from .views import PublishView urlpatterns = [ path(

"publish/", PublishView.as_view(), name="publish" ), ]
```

8. Permission Check in Templates

You can check permissions inside Django templates.

```
javascript.snippet JAVASCRIPT

{% if perms.blog.can_publish %} <a href="{% url 'publish' %}"> Publish
Article
    </a>{% endif %}
```

If the user has the permission, the link will be displayed.

Important

Hiding a button or link is **not a security mechanism**.

A user could still access the URL directly.

Therefore:

```
javascript.snippet JAVASCRIPT

Template UI control View/API Actual security enforcement
```

The permission should be enforced on the backend.

9. Admin Panel and Permissions

Django Admin allows you to manage:

```
javascript.snippet JAVASCRIPT
UsersGroupsPermissions
```

You can assign individual permissions to users.

For example:

```
javascript.snippet JAVASCRIPT
Faruk view_article add_article change_article
```

10. Groups

If many users need the same permissions, use a **Group**.

Example:

```
javascript.snippet JAVASCRIPT
Editor Group add_article change_article view_article
```

Then:

```
javascript.snippet JAVASCRIPT
Faruk Editor Group Editor Permissions
```

This makes permission management much easier.

11. User Permissions + Group Permissions

A user's effective permissions can come from both:

```
javascript.snippet JAVASCRIPT
User Permissions +Group Permissions Effective Permissions
```

Example:

```
javascript.snippet JAVASCRIPT
Editor Group view_article change_articleFaruk delete_article
```

Effective permissions:

```
javascript.snippet JAVASCRIPT
view_articlechange_articleddelete_article
```

12. is_staff, is_superuser, and has_perm()

These are not the same thing.

is_staff

```
javascript.snippet JAVASCRIPT
user.is_staff
```

This is primarily related to whether the user can access the Django Admin site.

is_superuser

```
javascript.snippet JAVASCRIPT
user.is_superuser
```

A superuser has all permissions by default.

has_perm()

```
javascript.snippet JAVASCRIPT
user.has_perm("blog.can_publish")
```

Checks whether the user has a specific permission.

Simple model:

```
javascript.snippet JAVASCRIPT

is_staff Admin accessis_superuser All permissionshas_perm() Specific permission
```

13. Permission Logic in `utils.py`

For more complex business rules, you can create a helper function.

```
javascript.snippet JAVASCRIPT

def can_access_publish(user):    return (    user.is_authenticated    and user.is_verified    and user.has_perm("blog.can_publish")    )
```

Then:

```
javascript.snippet JAVASCRIPT

if can_access_publish(request.user):    ...
```

However, the actual authorization should still be enforced at the appropriate backend/API/view layer.

14. Django Permission Flow

```
javascript.snippet JAVASCRIPT

User Login    Authentication    User Identity    Permission / Group Check
Authorization
    Allowed / Denied
```

Part 2: Race Condition

15. What is a Race Condition?

A **Race Condition** occurs when multiple concurrent operations access or modify the same shared data, and the result depends on the timing or order of those operations.

Simple idea:

javascript.snippetJAVASCRIPT

Multiple Requests Same Data Concurrent Access Unsafe Read Modify
Write
Race Condition

16. Simple Example

Suppose:

javascript.snippetJAVASCRIPT

balance = 100

Two threads are running:

javascript.snippetJAVASCRIPT

Thread A subtract B subtract 30

Expected:

javascript.snippetJAVASCRIPT

100 - 50 - 30 = 20

But both threads might read the old value:

javascript.snippetJAVASCRIPT

Thread A reads B reads 100 - 50 = 100 - 30 = 70

If B writes last:

javascript.snippetJAVASCRIPT

Final balance = 70

Expected:

```
javascript.snippet JAVASCRIPT
20
```

This is a classic race condition.

17. Causes of Race Conditions

Common causes include:

1. Concurrent execution

Multiple threads, processes, or requests execute at the same time.

2. Shared resources

For example:

```
javascript.snippet JAVASCRIPT
Database rowFileMemory variableInventoryWallet balanceCounter
```

3. Non-atomic operations

For example:

```
javascript.snippet JAVASCRIPT
value = obj.valuevalue -= .value = value
```

This is a read-modify-write sequence, not a single atomic database operation.

18. Race Condition in Django

Web applications can receive multiple requests at nearly the same time.

Example:

```
javascript.snippet JAVASCRIPT
Product stock = A buys B buys 5
```

If both requests read the stock before either update is saved, a race condition can occur.

19. Unsafe Django Code

```
javascript.snippet JAVASCRIPT

def buy_product(request, id):    product = Product.objects.get(id=id)    qty = int(
request
.POST.get("qty
"))    if product.stock >= qty:        product.stock -= qty        product.save()    return JsonResponse(
status": "ok"    })
```

The problem is:

```
javascript.snippet JAVASCRIPT

Request A    Reads stock = 10    Reads stock = 10 - 7 = 10 - 5 = 5
```

Final database value may become:

```
javascript.snippet JAVASCRIPT

5
```

But the two orders requested:

```
javascript.snippet JAVASCRIPT

7 + 5 = 12
```

while only 10 items were available.

20. Solution 1 -- select_for_update()

Django provides:

```
javascript.snippet JAVASCRIPT

select_for_update()
```

to lock selected database rows within a transaction.

Example:

```
javascript.snippet JAVASCRIPT

from django.db import transactionfrom django.http import JsonResponsefrom .models
import
Productdef buy_product(request, id):    qty = int(request.POST.get("qty
"))    with transaction.atomic():        product = (            Product.objects
stock"]        )        return JsonResponse({            "status": "success
"        })        return JsonResponse({            "
status": "out_of_stock"        })
```

.select_

21. How select_for_update() Works

Suppose:

```
javascript.snippet JAVASCRIPT

Stock = 10
```

Request A:

```
javascript.snippet JAVASCRIPT

select_for_update()    Database row locked
```

Request B tries to update the same row:

```
javascript.snippet JAVASCRIPT

Request B    Waits for the lock
```

After Request A finishes its transaction:

```
javascript.snippet JAVASCRIPT

Lock released    Request B continues    Reads the updated stock
```

This prevents the two transactions from independently modifying the same row based on the same stale value.

22. Why transaction.atomic()?

select_for_update() should normally be used inside a transaction.

```
with transaction.atomic():    product = Product.objects.select_for_update().get(...)
```

`atomic()` groups the database operations into a transaction.

If an exception occurs, Django can roll back the transaction.

23. Solution 2 -- F () Expression

For simple counter or stock updates, Django's `F ( )` expressions are very useful.

```
from django.db.models import F
updated = Product.objects.filter(id=id, stock__gte=qty).update(stock=F("stock") - qty)
```

Then:

```
if updated:    return JsonResponse({"status": "success"})
return JsonResponse({"status": "out_of_stock"})
```

24. How F () Works

The application does not first read the stock into Python.

Instead, the database performs the update using the current database value.

Conceptually:

```
UPDATE product SET stock = stock - qty WHERE id = ... AND stock >= qty;
```

This is useful for atomic field updates.

25. select_for_update () vs F ()

Feature	<code>select_for_update()</code>	<code>F()</code>
Main purpose	Row locking	Database-side expression
Transaction	Used inside transaction	Not always required
Complex business logic	Excellent	Limited
Simple counter update	Possible	Excellent
Reads object first	Yes	Usually no
Explicit row lock	Yes	Not the main purpose
Example	Stock + multiple checks	Stock decrement

Important

`F()` and `select_for_update()` are not always interchangeable.

Use `F()` when you need a simple atomic database update.

Use `transaction.atomic() + select_for_update()` when you need to read a row, perform multiple business-rule checks, and then safely update that row.

26. Other Race Condition Solutions

1. Database Transactions

```

javascript.snippet
JAVASCRIPT

from django.db import transaction
with transaction.atomic():
    ...

```

2. Row Locking

```

javascript.snippet
JAVASCRIPT

select_for_update()

```

3. Atomic Database Updates

```
javascript.snippet JAVASCRIPT
F("balance") - amount
```

4. Queue

Some operations can be processed sequentially:

```
javascript.snippet JAVASCRIPT
Request Queue Worker Process
```

Task queues such as Celery or RQ can be useful depending on the architecture.

5. Application-Level Lock

For Python threads:

```
javascript.snippet JAVASCRIPT
from threading import Lock
lock = Lock()
with lock:
    balance -= 50
```

However, this does **not** provide a shared lock across multiple Django worker processes or multiple servers.

For shared database state, database-level synchronization is generally more appropriate.

27. Database Backend Considerations

`select_for_update()` depends on the database's transaction and locking capabilities.

Therefore, you should understand the behavior of your production database.

For MySQL, a transactional storage engine such as **InnoDB** is important for transactional locking behavior.

28. Real-World Race Condition Examples

E-commerce Stock

```
javascript.snippet JAVASCRIPT

Stock = A Buy B Buy 1
```

Wallet

```
javascript.snippet JAVASCRIPT

Balance = A Withdraw B Withdraw 500
```

Ticket Booking

```
javascript.snippet JAVASCRIPT

Available seats = A BookUser B Book
```

Coupon

```
javascript.snippet JAVASCRIPT

Coupon usage limit = A RedeemUser B Redeem
```

Like Counter

```
javascript.snippet JAVASCRIPT

likes = requests likes + 1
```

Unsafe read-modify-write logic can cause lost updates.

29. Race Condition vs Deadlock

These are different problems.

Race Condition

```
javascript.snippet JAVASCRIPT
Concurrent Operations Timing Problem Incorrect / Unexpected Result
```

Deadlock

```
javascript.snippet JAVASCRIPT
Transaction A Lock X waits for YTransaction B Lock Y waits for X Both keep
waiting
```

So:

```
javascript.snippet JAVASCRIPT
Race Condition Incorrect concurrency behaviorDeadlock Transactions/processes become
stuck
waiting
```

30. Authentication vs Authorization vs Permission

This is very important for interviews.

```
javascript.snippet JAVASCRIPT
Authentication Who are you?Authorization What are you allowed to do?
Permission Specific authorization rule
```

Example:

```
javascript.snippet JAVASCRIPT
Login AuthenticationLogged-in User Has blog.can_publish? Authorization Allow / Deny
```

31. Final Mental Model Django Permission

```
javascript.snippet JAVASCRIPT

User Authentication User / Group Permission has_perm() Allow / Deny
```

Django Race Condition

```
javascript.snippet JAVASCRIPT

Multiple Requests      Same Database Row      Concurrent Read / Write      Race
Condition
      Synchronization      atomic()select_for_update()F()Database Constraints /
Atomic
Operations
```

32. Interview Quick Revision

What is Permission?

Authorization for a specific action.

What are Django's default model permissions?

```
javascript.snippet JAVASCRIPT

addchangeldeleteview
```

How do you check a permission?

```
javascript.snippet JAVASCRIPT

user.has_perm("app.permission")
```

How do you protect an FBV?

```
javascript.snippet JAVASCRIPT

@permission_required("app.permission")
```

How do you protect a CBV?

```
javascript.snippet JAVASCRIPT
PermissionRequiredMixin
```

What is a Group?

A way to manage a set of permissions for multiple users.

What is a Race Condition?

A concurrency problem where the timing/order of operations causes incorrect or unexpected results when shared data is accessed or modified.

How can you prevent race conditions in Django?

```
javascript.snippet JAVASCRIPT
transaction.atomic()select_for_update()F()Database constraintsAtomic database
operationsQueue
where appropriate
```

What is select_for_update()?

A Django ORM mechanism that requests row-level locking for selected rows within a transaction.

What is F()?

A Django ORM expression that allows database-side operations using the current value of a model field.

Is runserver a production server?

No. It is Django's development server.

Is hiding a button enough for permission security?

No. Permissions must be enforced on the backend/view/API as well.